PROJECT PROPOSAL

McKesson Medical-Surgical Solutions

Sales Intelligence and Analytics Platform

A Data-Driven Recommendation and Business Intelligence System
for Acute Care and Extended Care Sales Operations

MSBA Capstone Project
William and Mary
April 2026

1. Executive Summary

This proposal outlines the design and development of a Sales Intelligence and Analytics Platform for McKesson Medical-Surgical Solutions. The platform is built as a direct response to the business challenge described in the project brief: sales representatives visiting customers often engage with only a narrow subset of products that customers mention explicitly. This dynamic creates missed opportunities for cross-sell, upsell, and proactive category expansion.

The solution does not attempt to replicate or replace McKesson infrastructure. It is designed as the intelligence layer that sits above historical data and delivers two primary outputs: ranked, explainable product recommendation lists for sales representatives to use before and during customer visits, and a live administrative analytics dashboard that provides continuous visibility into customer behavior, product performance, segment health, and recommendation effectiveness.

The platform is grounded in five machine learning modules applied to two years of transaction-level sales data, customer profile data, and the full active product catalog. The architecture is designed to handle the scale of McKesson data responsibly, using a cloud object store for raw data, an analytical query engine optimized for large datasets, and a lightweight operational database for serving outputs. The result is a system that is both technically rigorous and directly aligned with the business objective of enabling more informed, proactive sales conversations.

2. Business Problem and Project Objectives

2.1 The Core Problem

McKesson Medical-Surgical Solutions serves more than 200,000 healthcare locations with a catalog exceeding 285,000 active products. Despite this scale, the typical sales interaction is reactive. Customers ask for a quote on a small number of products they already know and use. Sales representatives fulfill that request but have no structured, data-driven mechanism to surface additional relevant products the customer may need but has not thought to request.

This results in three measurable business costs. First, missed revenue from products the customer would have purchased if prompted. Second, lower customer satisfaction from gaps in supply chain coverage that could have been closed. Third, inefficient sales rep time spent on reactive quoting instead of consultative selling.

2.2 What McKesson Asked For

The project brief specifies two levels of analysis:

- **Level 1.** Business Segment Recommendations:

Identify and recommend products tailored to Acute Care and Extended Care segments based on aggregated purchase patterns.

- **Level 2.** Customer Level Personalization:

Cluster Acute Care and Extended Care customers into meaningful groups using behavioral and demographic data, then generate personalized product recommendations informed by individual purchase histories and cluster-level insights.

The stated deliverable is a mechanism to provide sales representatives with ready-to-go item lists that are relevant to each customer, enabling proactive suggestions that increase order size and customer satisfaction.

2.3 Project Objectives

1. Build a machine learning pipeline that processes two years of transaction, customer, and product data to generate ranked, personalized product recommendation lists for every Acute Care and Extended Care customer.
2. Cluster customers into meaningful behavioral segments and use those segments to enrich both customer-level recommendations and segment-level product gap analysis.
3. Surface these recommendation outputs in a sales representative interface that allows reps to look up a customer before a visit and review a prioritized pitch list with explanations.
4. Deliver a live administrative analytics dashboard that answers the key business intelligence questions McKesson cares about, updated continuously as new interactions are recorded.
5. Design the architecture to handle the real data scale of two years of transaction-level data across 200,000 healthcare locations without relying on fragile or undersized tooling.

3. System Architecture

The architecture is organized into four layers: data storage, analytical processing, machine learning, and application delivery. Each layer is selected based on the actual scale and nature of McKesson data rather than default assumptions.

3.1 Architecture Overview

Layer	Technology	Purpose
Data Storage	Cloudflare R2	Object store for raw McKesson data files. S3-compatible, free at project scale, retains original data securely without a managed database cost.
Analytical Engine	DuckDB	In-process analytical query engine that runs SQL directly against parquet files in R2. Handles hundreds of millions of rows on a single machine. No Spark cluster required.
Feature Store	PostgreSQL (Supabase)	Operational relational database for recommendation outputs, cluster assignments, user sessions, and dashboard aggregates. Stays small and fast because heavy computation happens in DuckDB.
Caching Layer	Redis	Stores pre-computed recommendation lists and live dashboard counters. Keeps the application interface fast without repeated database queries.
ML Processing	Python	pandas, DuckDB Python API, scikit-learn, LightGBM, mlxtend, implicit. All model training and inference happens in Python using features engineered via DuckDB.
Backend API	FastAPI	Serves recommendation endpoints, dashboard data endpoints, and real-time update channels. Async by design for live dashboard support.
Real-Time Updates	PostgreSQL LISTEN/NOTIFY + WebSockets	Replaces Kafka for event broadcasting. Delivers live updates to dashboard clients without the operational overhead of a Kafka cluster.
Frontend	React + Tailwind + Recharts	Two views: the sales representative pitch interface and the administrative analytics dashboard.

3.2 Why This Architecture Over the Initial Draft

The initial draft proposed PostgreSQL and pandas as the primary data processing stack. For a project with two years of transaction-level data from more than 200,000 healthcare locations, this creates a real risk of memory exhaustion and slow feature engineering pipelines. The revised architecture addresses this with two deliberate choices.

First, Cloudflare R2 stores the raw McKesson data files as parquet. This keeps the data in a format optimized for columnar analytical access and satisfies the NDA requirement for secure, controlled storage. Second, DuckDB queries those parquet files directly using SQL, pushing the

heavy aggregation work into a purpose-built analytical engine rather than loading full datasets into pandas memory. Once DuckDB produces the feature summaries, pandas handles the smaller downstream ML processing comfortably.

Apache Kafka has been removed entirely. The real-time dashboard requirements for this project do not justify the operational complexity of a Kafka cluster. PostgreSQL LISTEN/NOTIFY combined with WebSockets delivers the same live update behavior at a fraction of the setup and maintenance cost. This is a deliberate architectural decision, not a shortcut, and reflects the actual scale and timing requirements of a capstone demonstration.

4. Data Strategy and Engineering

4.1 Data Sources

McKesson will provide three datasets. All data will be stored in Cloudflare R2 upon receipt and converted to parquet format before any processing begins.

- Customer Data:

All Acute Care and Extended Care customers who have purchased from McKesson in the last two years. Includes customer type, location, and size fields used for segmentation and clustering.

- Item Data:

The full active McKesson Medical-Surgical catalog. Used for product feature engineering, catalog coverage analysis, and recommendation candidate generation.

- Sales Data:

Transaction-level purchase history for all Acute and Extended Care customers over the last two years. The primary input for all machine learning modules.

4.2 Data Scale Consideration

Two years of transaction-level data across more than 200,000 healthcare locations buying from a 285,000 product catalog represents a dataset that may contain tens of millions of rows at the order line level. The feature engineering pipeline is designed with this in mind. All heavy aggregations, joins, and intermediate transformations will be executed in DuckDB before any data is handed to scikit-learn or LightGBM. This ensures that model training operates on pre-aggregated feature tables rather than raw transaction rows.

4.3 Feature Engineering Pipeline

The feature engineering pipeline will produce two primary feature tables: one at the customer level and one at the product level. These tables feed all five machine learning modules.

Customer-level features include:

- Total spend over the two-year period
- Purchase frequency and average order value
- Recency of most recent purchase
- Number of unique products and unique categories purchased
- Category breadth ratio relative to segment peers
- Average inter-purchase interval by category
- Customer type, region, and size from the customer profile table

Product-level features include:

- Total units sold and total revenue
- Number of distinct customers who purchased the product
- Category and subcategory assignment

- Co-purchase frequency with other products
- Average reorder interval across customers who purchased multiple times

5. Machine Learning Modules

Five machine learning modules are implemented. Each module addresses a distinct business question and contributes to the final recommendation output. The modules are designed to be composable: outputs from upstream modules feed into downstream modules, producing a layered recommendation that is more accurate and more explainable than any single model alone.

5.1 Module 1: Customer Segmentation and Clustering

Purpose: Group Acute Care and Extended Care customers into meaningful behavioral segments that can inform both segment-level product gap analysis and personalized recommendation logic.

- Algorithms: K-Means with silhouette score validation, with HDBSCAN tested as an alternative for non-spherical cluster shapes
- Input features: RFM metrics (recency, frequency, monetary value), category breadth, purchase diversity, customer size and type, geographic region
- Output: Cluster assignments written to PostgreSQL, cluster personas labeled with interpretable names based on feature centroids
- Evaluation: Silhouette coefficient, Davies-Bouldin index, visual validation via PCA and UMAP projection

The cluster assignments are used throughout the rest of the system. Every customer recommendation is filtered and ranked against the behavior of peers in the same cluster. The admin dashboard uses cluster membership to surface segment health, migration trends, and product coverage gaps by persona.

5.2 Module 2: Collaborative Filtering

Purpose: Recommend products based on the purchase patterns of similar customers.

- Approach: Item-based collaborative filtering using implicit feedback signals derived from purchase frequency and quantity
- Library: implicit (GPU-optional ALS implementation optimized for implicit feedback matrices)
- Input: Customer-item interaction matrix built from two years of transaction data
- Output: Top-N product candidates per customer, scored by predicted affinity
- Evaluation: Precision at K, recall at K, and coverage across the catalog

Collaborative filtering handles the cold-start problem for products by leveraging the density of the two-year transaction history. Customers who share cluster membership with high-volume buyers benefit from richer recommendation signals even if their own purchase history is shorter.

5.3 Module 3: Association Rule Mining

Purpose: Identify products that are commonly purchased together and use these relationships to generate cross-sell suggestions.

- Algorithm: FP-Growth for computational efficiency on large transaction sets
- Metrics: Support, confidence, and lift. Rules with lift greater than 1.5 and confidence above 0.4 are retained
- Input: Transaction baskets constructed from the sales data at the order level
- Output: Product association rules stored in PostgreSQL, indexed by antecedent product for fast lookup

Association rules contribute directly to the sales representative pitch list. When a customer is known to purchase Product A, rules with A as the antecedent produce candidates for the pitch list that are ranked by lift. These suggestions carry a specific explanation tag: customers who buy this also tend to buy that. This is one of the most interpretable recommendation signals available to sales reps.

5.4 Module 4: Propensity and Purchase Likelihood Ranking

Purpose: Score each recommendation candidate for each customer and produce a final ranked pitch list.

- Algorithm: LightGBM gradient boosting classifier trained on historical purchase events
- Input features: Customer cluster membership, RFM metrics, product category match score, prior purchase overlap, association rule evidence, collaborative filtering score, days since last purchase of the category
- Output: A probability score for each customer-product pair representing the likelihood of purchase within the next sales cycle
- Evaluation: AUC-ROC, precision-recall curve, calibration curve

The propensity model serves as the ranking layer that combines outputs from all other modules. Every recommendation candidate, regardless of its source, passes through the propensity scorer. The final pitch list presented to a sales representative is sorted by propensity score descending, with the highest-confidence suggestions at the top.

5.5 Module 5: Reorder Signal Detection

Purpose: Identify products that a customer is likely due to reorder based on historical purchase intervals.

- Approach: Rule-based interval analysis in the first implementation. Average inter-purchase interval per customer-product pair is computed in DuckDB. Products where the elapsed time since the last purchase exceeds 80 percent of the historical average interval are flagged as reorder candidates.
- Enhancement: A time-series predictive layer using Prophet or lightweight ARIMA models will be added if development time permits, providing a smoother reorder probability that accounts for seasonal demand patterns.
- Output: Reorder flag and estimated days until next likely purchase, stored per customer-product pair in PostgreSQL

Reorder signals are surfaced with a distinct label in the sales representative pitch list, giving reps immediate visibility into supply continuity opportunities that are both high-priority for the customer and low-resistance from a sales perspective.

6. Recommendation Pipeline

The recommendation pipeline is the end-to-end process that converts raw data into a ranked, explainable pitch list for each customer. The pipeline runs in two modes: a batch mode executed nightly that precomputes recommendations for all customers, and an on-demand mode triggered when a sales representative looks up a specific customer.

6.1 Batch Pipeline Steps

6. DuckDB processes the raw transaction parquet files and produces updated customer and product feature tables.
7. The customer segmentation model assigns updated cluster labels to all customers.
8. The collaborative filtering model generates a candidate list of top-50 products per customer.
9. The association rule engine generates additional candidates for each customer based on their most recent purchases.
10. Reorder signal detection flags products eligible for reorder.
11. The propensity model scores all candidates and produces a final ranked list of the top 20 products per customer.
12. Ranked lists and explanation tags are written to PostgreSQL and cached in Redis.

6.2 Explanation Tags

Every product in a pitch list carries one or more explanation tags that tell the sales representative why the product was recommended. These tags are generated by a rules layer built on top of the model outputs.

- Frequently purchased together: The product co-occurs with the customer's recent purchases at high lift.
- Customers like you also buy: Collaborative filtering signal from peers in the same cluster.
- Due for reorder: The customer's reorder interval for this product has been exceeded.
- Category gap: Other customers in this segment regularly purchase this category but this customer does not.
- High propensity match: The ranking model scored this product in the top decile of purchase likelihood.

6.3 Sales Representative Interface

The sales representative view is a focused, single-purpose interface. A rep enters a customer name or ID and receives a profile summary alongside a ranked pitch list. The profile summary shows cluster assignment, purchase history highlights, and top categories. The pitch list shows the top 20 recommended products, sorted by propensity score, with explanation tags and a one-click export to CSV for use in quote preparation.

The interface is intentionally simple. Its value is in the quality and relevance of the recommendations, not in feature complexity. A sales rep should be able to review the list in under two minutes before a customer visit.

7. Administrative Analytics Dashboard

The administrative analytics dashboard is the central business intelligence interface of the platform. It is designed to answer the questions that McKesson management would ask in a weekly business review, a quarterly planning session, or an ad-hoc investigation of sales performance. Every panel is connected to live data and updates continuously as new interactions are recorded.

The dashboard is organized into six analytical domains, each with its own view.

7.1 Sales Performance

- Total revenue by time period with selectable granularity: daily, weekly, monthly, and quarterly
- Revenue breakdown by business segment: Acute Care versus Extended Care
- Revenue breakdown by geographic region and customer size tier
- Top 20 products by revenue, by volume, and by growth rate
- Sales representative performance: revenue generated, number of accounts touched, recommendation conversion rate per rep
- Period-over-period comparison with percentage change indicators

7.2 Customer Intelligence

- Cluster composition: number of customers per persona, average spend per cluster, geographic distribution of each cluster
- Cluster migration tracker: visualization of customers who have moved between clusters across rolling time windows, indicating whether the customer base is upgrading or at risk
- Churn risk indicators: customers with a declining purchase frequency trend flagged for sales rep attention
- New versus returning customer breakdown by segment and time period
- Average customer lifetime value by cluster and segment

7.3 Product and Category Intelligence

- Category performance heatmap: revenue and volume by product category across segments
- White space analysis: product categories that a given cluster purchases at significantly lower rates than comparable clusters, surfacing growth opportunities
- Product velocity tracker: products with accelerating or decelerating purchase frequency
- Catalog coverage: what percentage of the active catalog is being purchased by each segment
- Market basket heatmap: which product pairs co-occur most frequently across the customer base

7.4 Recommendation Intelligence

- Recommendation coverage: how many customers received a pitch list in the last 30 days

- Precision at K metrics by segment and by ML module, showing which recommendation source performs best
- Recommendation conversion rate: how often a pitched product appears in the customer's next order
- Top converting products from recommendations versus organic orders
- Module attribution: what fraction of converted recommendations came from collaborative filtering versus association rules versus reorder signals

7.5 Basket and Order Analysis

- Average basket size over time by segment and by sales representative
- Order frequency distribution: histogram of purchase intervals across the customer base
- Cross-sell effectiveness: orders that included at least one recommended product versus baseline
- Category co-purchase network: interactive graph showing the strongest product category relationships in the transaction data

7.6 Live Activity Feed

- Real-time counter of orders placed today and revenue generated today, updated via WebSocket
- Recent recommendation pitch list requests by sales representatives
- New customer registrations and cluster assignments as they are processed
- Reorder alert volume: number of customers flagged for reorder today

All dashboard panels support filtering by date range, business segment, geographic region, customer cluster, and sales representative. Any combination of filters can be applied simultaneously, and the dashboard updates all panels in response. This allows a business analyst or sales manager to move from a high-level revenue summary to a specific segment's product gap analysis to an individual customer's recommendation history within the same interface without switching tools.

8. Machine Learning Module to Feature Mapping

Platform Feature or Business Question	ML Module or Data Source
Customer personas and segment profiles	Customer Segmentation and Clustering
Personalized product pitch list for sales reps	Collaborative Filtering + Association Rules + Propensity Ranker
Cross-sell suggestions based on current basket	Association Rule Mining
Products due for reorder	Reorder Signal Detection
Final ranked pitch list ordering	Propensity and Purchase Likelihood Model
Explanation tags for each recommendation	Rules layer over all module outputs
White space analysis in admin dashboard	Segmentation + Catalog Coverage Analysis
Cluster migration trends	Segmentation applied over rolling time windows
Churn risk flags	RFM feature trends from DuckDB feature pipeline
Recommendation conversion analytics	Logged outputs from PostgreSQL recommendation store
Market basket heatmap	Association Rule Mining co-occurrence matrix
Category gap analysis by segment	Segmentation + DuckDB purchase frequency aggregates

9. Technology Stack

9.1 Frontend

Framework: React

- React Router for view routing between sales rep and admin roles
- Axios for API communication with the FastAPI backend
- Zustand for lightweight client-side state management
- Tailwind CSS for layout and styling
- Recharts and Nivo for dashboard chart components
- WebSocket client for live dashboard updates

9.2 Backend

Framework: FastAPI

- Pydantic for request and response validation
- SQLAlchemy and Alembic for PostgreSQL ORM and schema migrations
- python-jose for JWT authentication and route protection
- Uvicorn as the ASGI server
- asyncpg for async PostgreSQL communication and LISTEN/NOTIFY event handling

9.3 Machine Learning

Language: Python 3.11

- DuckDB for all SQL-based feature engineering over parquet files
- pandas for downstream feature manipulation at reduced data volumes
- scikit-learn for clustering, preprocessing, and evaluation utilities
- LightGBM for the propensity ranking model
- mlxtend for FP-Growth association rule mining
- implicit for ALS-based collaborative filtering
- joblib for model serialization and batch inference
- UMAP and matplotlib for cluster visualization and evaluation outputs

9.4 Data Storage and Infrastructure

- Cloudflare R2: raw data lake for McKesson-provided files in parquet format
- PostgreSQL on Supabase free tier: operational database for recommendations, cluster assignments, sessions, and aggregates
- Redis: cache for pre-computed recommendation lists and live dashboard counters
- DuckDB: in-process analytical engine querying R2 parquet files via the S3-compatible API
- Docker Compose: local orchestration of FastAPI, PostgreSQL, and Redis for development and demo
- GitHub: version control and project task tracking

9.5 Deployment for Capstone Demo

- Frontend deployed on Vercel
- Backend deployed on Render free tier
- PostgreSQL on Supabase free tier
- Redis on Upstash free tier
- DuckDB runs in-process within the FastAPI backend, querying R2 via HTTP
- All McKesson data remains in Cloudflare R2 with access keys restricted to the project

10. Database Design

10.1 Operational Tables in PostgreSQL

- users: platform accounts with role assignment
- customer_profiles: customer attributes, cluster assignment, segment classification
- products: active catalog items with category, subcategory, and pricing metadata
- recommendation_outputs: the full ranked pitch list per customer with explanation tags and propensity scores
- recommendation_logs: record of which recommendations were served to which sales rep and when
- recommendation_conversions: record of when a pitched product appeared in a subsequent customer order
- reorder_signals: customer-product pairs flagged for reorder with estimated reorder date
- cluster_assignments: current and historical cluster labels per customer
- dashboard_aggregates: pre-computed summary metrics refreshed by the nightly batch pipeline
- audit_logs: record of all admin and sales rep actions for security and compliance

10.2 Analytical Tables Produced by DuckDB

These tables are written to PostgreSQL by the nightly batch pipeline after DuckDB processing completes.

- customer_features: full RFM and behavioral feature vector per customer
- product_features: sales velocity, customer reach, and category metrics per product
- customer_item_interactions: interaction matrix used as input to collaborative filtering
- association_rules: mined product co-purchase rules with support, confidence, and lift
- daily_sales_aggregates: pre-rolled revenue and volume summaries by date, segment, and category
- category_coverage_by_cluster: what fraction of each product category each cluster purchases

11. Security and NDA Compliance

All McKesson-provided data is treated as confidential in accordance with the NDA and Copyright License Agreement. The following measures are in place throughout the project.

11.1 Data Security

- All raw McKesson data files are stored exclusively in Cloudflare R2 with private access. Bucket access is restricted to project credentials held by the project team.
- No raw McKesson data is stored on local personal devices beyond temporary development sessions. All persistent storage occurs in R2.
- PostgreSQL on Supabase is accessed only via environment variables stored in a private GitHub repository with access controls. No credentials are committed to source code.
- Upon project completion, all McKesson data will be permanently deleted from R2 and all other storage locations in accordance with NDA terms.

11.2 Application Security

- JWT-based authentication with access tokens and refresh tokens for all protected API routes
- Role-based access control distinguishing admin users from sales representative users
- Password hashing using bcrypt via passlib
- HTTPS enforced in all deployed environments
- CORS configuration restricting API access to the frontend domain
- Input validation on all API endpoints via Pydantic schemas

12. 22-Day Execution Plan

The following plan assumes eight focused working hours per day. The plan is sequenced so that the data pipeline and ML core are completed before the application layer, ensuring the interface is built around proven outputs rather than placeholder data.

Days 1 to 2: Foundation and Data Ingestion

- Set up Cloudflare R2 bucket, load McKesson CSV files, convert to parquet
- Initialize GitHub repository, Docker Compose configuration, and project board
- Initialize FastAPI project structure and React project structure
- Configure PostgreSQL on Supabase and run initial schema migrations
- Validate DuckDB connection to R2 parquet files with exploratory queries

Days 3 to 4: Feature Engineering Pipeline

- Build DuckDB SQL pipeline for customer-level feature extraction: RFM, category breadth, purchase diversity, inter-purchase intervals
- Build product-level feature extraction: velocity, customer reach, co-purchase frequency
- Write customer and product feature tables to PostgreSQL
- Exploratory data analysis: distributions, outliers, segment breakdowns

Days 5 to 6: Customer Segmentation

- Train K-Means clustering on customer feature vectors with silhouette score validation
- Test HDBSCAN as an alternative and compare cluster quality metrics
- Label cluster personas based on feature centroids
- Write cluster assignments to PostgreSQL customer_profiles table
- Generate UMAP visualizations for cluster validation and eventual dashboard use

Days 7 to 8: Association Rule Mining and Collaborative Filtering

- Build transaction basket dataset from sales data using DuckDB
- Run FP-Growth with support and confidence thresholds, filter by lift
- Write association rules to PostgreSQL
- Build customer-item interaction matrix
- Train ALS collaborative filtering model using implicit library
- Generate top-50 recommendation candidates per customer and write to PostgreSQL

Days 9 to 10: Propensity Model and Reorder Signals

- Engineer propensity model training dataset combining all upstream signals
- Train LightGBM classifier, tune hyperparameters, evaluate AUC-ROC and precision-recall
- Compute reorder signals using inter-purchase interval analysis in DuckDB

- Build end-to-end recommendation pipeline: candidate generation, scoring, ranking, explanation tagging
- Write final ranked pitch lists and explanation tags to PostgreSQL

Days 11 to 12: FastAPI Backend

- Build authentication endpoints: login, token refresh, route protection middleware
- Build recommendation endpoints: fetch ranked pitch list by customer ID
- Build dashboard data endpoints for all six analytical domains
- Implement PostgreSQL LISTEN/NOTIFY listener and WebSocket broadcast for live updates
- Configure Redis caching for recommendation lists and dashboard counters

Days 13 to 16: Admin Analytics Dashboard

- Day 13: Sales performance panels. Revenue by time period, segment, region. Rep performance table.
- Day 14: Customer intelligence panels. Cluster composition, migration tracker, churn risk.
- Day 15: Product intelligence panels. Category heatmap, white space analysis, market basket heatmap.
- Day 16: Recommendation intelligence panels, live activity feed, global filter controls.

Days 17 to 18: Sales Representative Interface

- Customer lookup and profile summary view
- Ranked pitch list display with explanation tags and propensity scores
- CSV export for quote preparation
- Reorder alert indicators in the pitch list

Days 19 to 20: Integration, Polish, and Demo Data

- End-to-end integration testing across all components
- Performance validation of dashboard load times and WebSocket latency
- Seed demo scenarios: a high-value Acute Care customer with rich history, a low-activity Extended Care customer showing white space opportunity, a cluster with strong reorder signal
- UI polish and navigation refinement

Days 21 to 22: Documentation and Presentation

- Architecture diagram and system documentation
- ML evaluation report: silhouette scores, precision at K, AUC-ROC, lift values, association rule quality metrics
- Final project report with business impact framing
- Presentation slide deck aligned to McKesson's stated business objectives
- End-to-end demo rehearsal and recording

13. Success Metrics and Evaluation

13.1 Machine Learning Metrics

Module	Primary Metric	Secondary Metric
Customer Segmentation	Silhouette Coefficient	Davies-Bouldin Index
Collaborative Filtering	Precision at 10 and Precision at 20	Catalog Coverage
Association Rule Mining	Lift (target above 1.5)	Confidence and Support
Propensity Ranker	AUC-ROC	Precision-Recall Curve
Reorder Detection	Recall of actual reorder events	Mean days error to actual reorder

13.2 Business Metrics

- Average number of products in a pitch list that appear in the customer's historical purchase categories, demonstrating relevance
- Proportion of pitch list items that represent products the customer has not purchased before, demonstrating incremental opportunity
- White space coverage: proportion of catalog gaps per customer that the system identifies
- Dashboard load time under three seconds for all standard panels
- WebSocket live update latency under two seconds from order event to dashboard update

14. Risks and Mitigation

Risk	Likelihood	Mitigation
Data volume causes pandas memory issues during feature engineering	Medium	DuckDB handles all heavy aggregation before pandas is invoked. Feature tables passed to ML models are pre-aggregated and small.
LightGBM propensity model underfits due to sparse interaction data	Low-Medium	Fall back to logistic regression baseline or use collaborative filtering scores directly as the ranking signal.
Collaborative filtering produces poor recommendations for low-activity customers	Medium	Use cluster-level purchase patterns as a fallback signal for customers with fewer than 5 historical orders.
22-day timeline is tight for full five-module ML pipeline	Medium	Prioritize modules 1, 2, and 3. Modules 4 and 5 are additive enhancements, not

		blockers for a functional recommendation list.
DuckDB R2 connectivity adds latency to the batch pipeline	Low	Run batch pipeline on a local DuckDB instance during development. Use R2 as the canonical store but allow local parquet copies for speed.
McKesson data arrives late or in unexpected format	Low	Build feature engineering pipeline against a synthetic dataset of comparable structure immediately so the ML and application layers are not blocked.

15. Final Recommendation and Closing Statement

This project is positioned as a sales intelligence platform for medical-surgical products, not a commercial application replacement. The architecture decisions reflect the actual scale of McKesson data, the real constraints of a 22-day timeline, and the genuine business objective of equipping sales representatives with proactive, data-driven guidance.

The administrative dashboard is designed to be a genuine analytical instrument. Every panel answers a specific business question. The filtering system allows a user to move from macro revenue trends to micro customer-level product gaps without switching contexts. The live update capability ensures the dashboard reflects current activity, not yesterday's snapshot.

The machine learning pipeline is composable and evaluable. Each module produces measurable outputs that can be assessed independently. The propensity ranking layer synthesizes all signals into a single ranked list, making the final recommendation output both principled and interpretable. The explanation tag system ensures sales representatives understand and trust what they are being shown.

The technical foundation, DuckDB over parquet on R2 feeding into PostgreSQL for serving, is chosen specifically for the data scale McKesson will provide. It avoids the operational risk of loading full two-year transaction histories into memory while maintaining the simplicity appropriate for a capstone project. The same architecture would scale to production use with minimal changes.

The outcome of this project will be a system that McKesson management can open during a sales strategy meeting, ask any reasonable question about their Acute and Extended Care customer base, and receive a clear, data-driven answer. That is the standard this project is built to meet.

Submitted in partial fulfillment of the MSBA Capstone Project requirements
William and Mary | April 2026
Prepared for McKesson Medical-Surgical Solutions